

LIBRARY MANAGEMENT SYSTEM

Course Code : CSE282

Course Title : Introduction to Programming
Language II (Java) Lab

Department : CSE

Faculty Name : Mr. Md. Ayon Mia

Submission Date : 27/09/26

Submitted By

Member 1 – SANJIDA ROHMAN
2023000000057

Member 2 – MD MUHIMINUL ISLAM ESTY
2024000000199

Member 3 – MD MAHFUZ KABIR JOY
2024200000669

Member 4 – MAHABUBUL ALAM PRITOM
2024000000176

Member 5 – AZIZUL HAQUE
2024200000143

1. INTRODUCTION

The Library Management System (LMS) is a Java-based desktop application, built with Swing and Object-Oriented Programming principles, that automates the core operations of a library. It manages the book catalogue (including non-circulating reference books), member records, issuing and returning of books, and reporting, replacing manual, paper-based record-keeping with a single organized system.

2. OBJECTIVES

What the project aims to achieve:

The project aims to build a complete, working library management application in Java that models real library entities and operations through a clean class hierarchy, so that book, member, and transaction records can be managed reliably from one system.

Specific goals of the project:

Design a class hierarchy that reflects real-world library entities, including the distinction between circulating and reference-only books.

Separate data entities, business logic, persistence, and presentation into distinct responsibilities.

Automate issuing and returning of books with accurate record-keeping.

Apply encapsulation, inheritance, abstraction, and polymorphism throughout the codebase.

Handle invalid operations gracefully through custom exceptions.

Generate usable reports on books, members, and issue activity.

Key functionalities to be implemented:

- 1. Add, update, search, and view books and reference books**
- 2. Register, update, and search members**
- 3. Issue and return books with record tracking**
- 4. Persist book and member data to files between sessions**
- 5. Generate reports on inventory, members, and issued or returned books**
- 6. A Swing GUI covering login, dashboard, book, member, issue, return, and report screens**

3. MOTIVATION

Manual library record-keeping is slow, error-prone, and hard to audit a librarian tracking issued books and due dates on paper or in spreadsheets can easily lose track of overdue items or double-issue a copy. Small libraries, department libraries, and campus reading rooms often lack the budget for a full commercial library system. This project was motivated by the real-world need for a lightweight, self-contained system that a small institution could run locally, while giving the team hands-on practice applying core Java and OOP concepts, including class hierarchies, file persistence, and custom exception handling to a problem with genuine everyday relevance.

4. APPLICATIONS

Where the project can be used:

1. School and college libraries
2. Departmental or research lab book collections
3. Community and public reading rooms
4. Private or personal book lending collections

Potential users of the system:

1. Librarians or staff who log in to manage the catalogue and members
2. Library members whose borrowing records are maintained by the system
3. Institution administrators reviewing inventory and activity reports

Practical uses and benefits:

1. Eliminates manual ledger-based tracking of borrowed books
2. Reduces lost or unaccounted-for books through consistent record-keeping
3. Keeps reference books clearly separated from circulating stock
4. Gives staff quick, report-based visibility into inventory and issue activity

5. PROJECT DESCRIPTION

Overall description of the system diagram:

The application follows a layered architecture, separating the GUI, business logic (manager classes), model entities, and file-based storage:

Swing GUI Layer (Login, Dashboard, Book, Member, Issue, Return, Report panels)

Manager or Service (BookManager, MemberManager, IssueManager, ReportManager)

Model (LibraryEntity, Book, ReferenceBook, Member, IssueRecord)

FileManager (Persistence)

Main features and functionalities:

- 1. Book catalogue management, including reference-only books that cannot be issued**
- 2. Member registration and record-keeping**
- 3. Issuing and returning books with full transaction history**
- 4. File-based persistence so data survives between sessions**
- 5. Custom exception handling for invalid operations**
- 6. Report generation for inventory, members, and issue activity**
- 7. A complete Swing GUI tying all modules together**

Major modules/classes of the system:

LibraryEntity (abstract)

- 1. Common base class for library holdings fields such as id, title, author; declares abstract behavior (e.g. displayDetails()).**

Book extends LibraryEntity

- 2. A standard circulating book adds fields such as isbn and copiesAvailable, and can be issued to members.**

ReferenceBook extends Book

- 3. A reference only book that overrides issuing behavior so it can be viewed but never checked out.**

BookManager

- 4. Business logic for adding, removing, searching, and updating books and reference books.**

Member

1. Represents a library member — fields such as memberId, name, contact details, and borrowing history.

MemberManager

2. Business logic for registering, updating, and searching members.

FileManager

3. Handles reading and writing book, member, and issue data to files so records persist between sessions.

IssueRecord

4. Represents a single borrowing transaction — fields such as recordId, book, member, issueDate, dueDate, and returnDate.

IssueManager

5. Business logic for issuing and returning books, including availability checks and updating IssueRecord entries.

Custom Exceptions

6. Dedicated exception classes (e.g. BookNotFoundException, MemberNotFoundException, BookUnavailableException, DuplicateEntryException) used across the manager classes.

ReportManager

7. Generates reports such as current inventory, registered members, and issued/returned book activity.

GUI Panels (Swing)

8. Login, Dashboard, Book, Member, Issue, Return, and Report panels, integrating all manager classes into one application.

Workflow of the system:

1. Staff log in through the Login panel, landing on the Dashboard.
2. BookManager and MemberManager load existing records at startup via FileManager.
3. Staff add/search books (Book or ReferenceBook) and register members through the Book and Member panels.

4. On the Issue panel, IssueManager checks availability (raising BookUnavailableException for reference books or fully issued titles) and creates an IssueRecord.
5. On the Return panel, IssueManager updates the matching IssueRecord with the return date.
6. FileManager persists all changes; ReportManager compiles inventory, member, and issue-activity reports for the Report panel.

Java/OOP concepts used in the project:

Encapsulation:

Class attributes are declared private across LibraryEntity, Book, ReferenceBook, Member, and IssueRecord, with controlled access through getters/setters.

Inheritance:

LibraryEntity

Book

ReferenceBook

Abstraction:

LibraryEntity is declared abstract, defining shared properties and behavior for all library holdings while leaving specifics to Book and ReferenceBook.

Polymorphism:

ReferenceBook overrides Book's issuing behavior so BookManager and IssueManager can treat every catalogue item as a LibraryEntity/Book while each subclass responds differently when an issue is attempted.

Exception Handling:

Custom exceptions raised by BookManager, MemberManager, and IssueManager are caught by the GUI layer and shown to the user as clear error messages rather than crashing the application.

6. TOOLS & TECHNOLOGIES

Programming language and version:

Java (JDK 17)

IDE used for development:

IntelliJ IDEA / VS Code

Libraries/frameworks used:

- 1. Java Swing – graphical user interface (Login, Dashboard, Book, Member, Issue, Return, Report panels)**
- 2. Java Collections Framework – in-memory management of books, members, and issue records**
- 3. Java I/O (File handling, via FileManager) – persistent storage of records between sessions**
- 4. Add any additional libraries actually used, JDBC, JUnit**

7. OUTPUT / RESULTS

Screenshots of the developed system:

[screenshot of Login panel here]

Sample inputs and outputs:

Error handling/output:

8. CONTRIBUTION

The project was divided among five group members so that each member could focus on specific modules.

Member	Responsibility	Main Work
Member 1	OOP Architecture + Book Management	LibraryEntity, Book, ReferenceBook, BookManager
Member 2	Member Management + File Persistence	Member, MemberManager, FileManager
Member 3	Issue & Return Management	IssueRecord, IssueManager
Member 4	Exceptions + Reports	Custom exceptions, ReportManager
Member 5	Swing GUI + Integration	Login, Dashboard, Book, Member, Issue, Return and Report panels

Tasks completed by each member:

1. Member 1: Designed the core OOP architecture and implemented the book-related class hierarchy and its manager.
2. Member 2: Implemented member record-keeping and the file-based persistence layer.
3. Member 3: Implemented the issue/return workflow and transaction records.
4. Member 4: Implemented the custom exception classes and the reporting logic.
5. Member 5: Built the Swing GUI and integrated all manager classes into a working application.

Modules/features developed by each member:

- 1. Member 1 – Book Management Module:**
LibraryEntity, Book, ReferenceBook, BookManager.
- 2. Member 2 – Member Management Module:**
Member, MemberManager, FileManager.
- 3. Member 3 – Issue & Return Module:**
IssueRecord, IssueManager.
- 4. Member 4 – Exception & Reporting Module:**
custom exceptions, ReportManager.
- 5. Member 5 – GUI & Integration Module:**
Login, Dashboard, Book, Member, Issue, Return, and Report panels.

9. CONCLUSION

Summary of the completed project:

The Library Management System was designed and implemented in Java using Object-Oriented Programming principles, covering book cataloguing (including reference only books), member management, issuing and returning, file-based persistence, custom exception handling, and reporting, all tied together with a Swing GUI.

Whether the objectives were achieved:

All planned objectives were achieved: the system supports a clear class hierarchy for library holdings, dedicated manager classes for each responsibility, automated issue/return tracking, persistent storage, and report generation, built on encapsulated, inheritance-based, and exception-safe design.

Limitations of the project:

- 1. Data is stored in local files via FileManager rather than a full database, limiting scalability for very large catalogues.**
- 2. No multi-user or networked/concurrent access support.**
- 3. No automatic fine calculation for overdue books in the current version.**

Possible future improvements:

- 4. Migrate storage to a relational database .**
- 5. Add automatic overdue-fine calculation and notifications.**
- 6. Build a web-based or networked client-server version for multi-user access.**
- 7. Add barcode/QR-code scanning for faster check-in/check-out.**

GITHUB REPOSITORY LINK :

<https://github.com/pritom176/Library-Management-System>